Recursion

Recursion is a technique where a function calls itself to solve a smaller instance of the same problem. The key insight: a big problem is expressed in terms of smaller versions of itself.

Factorial Example

$n! = n * (n-1)!$, with base case $1! = 1$

```
fact(n):
  if n ≤ 1: return 1
  return n * fact(n - 1)
```

Recursion tree for fact(4):

{"type": "tree", "nodes": "4:3:- 3:2:- 2:1:- 1:-:-"}

Each call spawns a smaller call until we hit the base case (n=1), then results propagate back up: 1 → 2 → 6 → 24.

Two Essential Parts

Every recursive function has two parts:

1. Base case - the simplest input with a known answer; stops recursion.
2. Recursive case - calls itself on a strictly smaller input, moving toward the base case.

Without a base case, recursion never terminates → stack overflow!

Call Stack: fact(+)

As calls are made, frames pile onto the call stack. When the base case returns, frames unwind:

Unwinding: fact(2) = 2*1 = 2, fact(3) = 3*2 = 6, fact(4) = 4*6 = 24 ✓

Fibonacci: The Danger of Repeated Work

Naive definition: fib(n) = fib(n−1) + fib(n−2), with fib(0)=0, fib(1)=1

{"type": "tree", "nodes": "F4:F3:F2a F3:F2b:F1a F2a:F1b:F0 F2b:-:- F1a:-:- F1b:-:- F0:-:-"}

Notice: fib(2) is computed twice (F2a and F2b)! For larger n, this explodes – naive fib has time complexity $O(2^n)$. Fix: memoization or iterative bottom-up DP → $O(n)$.

Tail Recursion

In standard recursion, work happens AFTER the recursive call returns (e.g., the * n in factorial). Tail recursion places the recursive call as the LAST operation, using an accumulator:

```
factTail(n, acc):
  if n ≤ 1: return acc
 return factTail(n - 1, n * acc)
```

Some compilers optimize tail recursion into a loop - no extra stack frames. This means O(1) space instead of O(n).

Recursion vs Iteration

Use recursion when:

* Problem is naturally recursive (trees, graphs, divide + conquer)
* Backtracking / exhaustive search (maze, permutations)
* Code clarity outweighs overhead

Use iteration when:

* Simple linear processing (sum, product, counting)
* Stack depth could be large (risk of overflow)
* Performance is critical and the recursive structure adds overhead

Rule of thumb: if the recursive step is trivial (just a loop in disguise), prefer iteration.

Every recursive solution needs a base case - without it, the function call